

Supplementary Material for: A Survey of Symbol Name Recovery in Software Reverse Engineering

HONGCHENG FAN, Nanjing University, China

JIELUN WU, Nanjing University, China

XINCHENG HE, Nanjing University, China

YANG FENG, Nanjing University, China

YIBIAO YANG, Nanjing University, China

BAOWEN XU, Nanjing University, China

QINGKAI SHI, Nanjing University, China

5 Evaluation Methods

5.1 Datasets

Table 1. Literature with **Unavailable** Data Set and Metrics for Evaluation

Literature	Dataset					Opt	Metrics
	Data Source	# Projs	# Files	# Funcs	Architecture		
UNSTRIP[14]	Glibc datasets, with different gcc and glibc versions.	13	-	-	-	O0-O3	P,R
DREAM++[34]	Example decompiled code from the three decompilers.	-	-	-	-	-	O
Debin[11]	Linux Packages, built with default compilation settings.	830	3.0K	237.0K	x86,x64,arm	O0-O3	P,R,F1
Punstrip[23]	1. Linux packages, compiled with gcc and clang.	5	149	-	x64	O0-O2	P,R,F1
	2. Debian packages.	14.0K	17.5K	-	x64	O0-O2	
Jaffe et al.[15, 16]	Github projects, compiled with gcc.	20.2K	174.4K	-	-	O0	A
Lacomis et al.[19]	Github projects, compiled with gcc.	20.0K	-	-	-	O0	A
SEAM[20]	1. Code generated by cfile, compiled with gcc 6.5.0.	-	-	1.6M	x64	O0	P,O
	2. Github algorithm code, compiled with gcc6.5.0.	-	-	20.0K	x64	O0	
Cfg2Vec[35]	ALLSTAR binaries with more then one arch.	1.1K	27.6K	1.4M	amd,arm,i386,mips	-	P
LmPa[33]	Randomly selected projects, compiled with Clang 12.	6	-	1.3K	-	O0	P,R,O
A2N[32]	Combination of Nero and Coreutils binary files.	-	753.5K	-	-	-	P,R,F1
BinAdapter[21]	AsmDepictor dataset, BinKit[17], and SPEC2006[9].	-	4.3K	760.5K	arm,x64,mips	O2	P,R,F1,O
LLASM[27]	Combination of BinaryCorp [31] and Debin datasets.	-	69.5K	1.6M	-	-	P,R,F1
BETAC[5, 6]	1. Juliet Test Suite and NDSS18, compiled with gcc.	-	97.5K	98.4K	x86	-	A,R,P,O
	2. Ubuntu packages.	50.0K	25.3K	25.4K	amd64	-	
	3. programs with three obfuscation techniques.	-	-	3.7K	-	-	

(Continued)

Table 1. Continued

Literature	Dataset					Opt	Metrics
	Data Source	# Projs	# Files	# Funcs	Architecture		
ADAM[8]	No evaluation.	-	-	-	-	-	-
Anti-ProGuard[2]	F-Droid apps.	-	-	-	-	-	A
Isobe et al.[12, 13]	Maven central repository (MCR) [24] programs.	-	17.7K	935.8K	-	-	P,R
Otsuki et al.[22]	No evaluation.	-	-	-	-	-	-
DeGuard[4]	F-Droid apps, exclude obfuscated.	1.8K	-	-	-	-	A
JSNice[25, 26]	Github projects, exclude obfuscated.	10.6K	327.2K	-	-	-	A
JSNaughty[30]	Github projects, exclude obfuscated, file-level dedup.	100.0K	735.9K	-	-	-	P
Context2Name[3]	Files from JS150, exclude obfuscated.	-	98.0K	-	-	-	A
Pigeon[1]	Github projects, file-level dedup.	26.6K	2.8M	-	-	-	A
JSNeat[29]	Github projects, exclude obfuscated, file-level dedup.	12.0K	322.0K	967.0K	-	-	A
DOBF[18]	Github projects, file-level dedup.	-	-	-	-	-	A,P,R,F1
Seq2Seq[28]	Files from JS150, file-level dedup.	-	98.0K	-	-	-	A

5.3 Artifact Evaluation

Table 2. Experience of Artifact Evaluation

Literature	Experimental Discoveries
Debin	1. Debin provides an online tool [10], but it returned a “502 Bad Gateway” error. 2. Debin’s performance drops clearly on our self-constructed dataset. 3. Most correctly recovered names are single characters in the test file provided in the repository. 4. Debin prefers to predict long names.
DIRE	1. Replication results match those reported in the paper. 2. DIRE’s performance drops significantly on our self-constructed dataset.
Nero	1. Replication results match those reported in the paper. 2. We failed to run the comparison evaluation because Nero depends on IDA Pro, which no longer supports Python 2.
Punstrip	We failed to run the artifact. As noted in the README, “Some scripts may be broken, this is a partial release with some latest research advancements held back.”
DIRECT	1. Replication results match those reported in the paper. 2. DIRECT uses the exact dataset from DIRE and does not include any data preprocessing steps. 3. DIRECT’s performance drops significantly on our self-constructed dataset.
Artuso et al.	1. The evaluation results match those presented in the paper. 2. The model’s performance drops significantly on our self-constructed dataset. 3. The implementation quality is low and differs from what is described in the paper. 4. The model prefers to predict short names.
SymLM	We failed to run the artifact as the trained model provided is not matched to code.

(Continued)

Table 2. Continued

Literature	Experimental Discoveries
DIRTY	1. DIRTY provides an online tool [7], but it only shows pre-stored examples. 2. Replication results match those reported in the paper. 3. DIRTY's performance drops clearly on our self-constructed dataset.
XFL	1. XFL's performance drops clearly on our self-constructed dataset. 2. XFL predicts only compound-word names, with the default top five most probable subtokens.
Cfg2Vec	1. We failed to run the artifact as the provided model download link is invalid. 2. The data preprocessing code is incomplete.
AsmDepictor	1. Replication results match those presented in the paper. 2. We failed to run the comparison evaluation because the data preprocessing code is incomplete.
BinAdapter	We failed to run the artifact because the authors did not release the data and trained model.
Epitome	We failed to run the artifact because it references an undefined attribute and lacks required files.
NER	1. Replication results match those presented in the paper. 2. NER's performance drops clearly on our self-constructed dataset.
VarBERT	1. Replication results match those presented in the paper. 2. VarBERT's performance drops clearly on our self-constructed dataset. 3. VarBERT produces very few examples because Joern often fails to analyze C-like decompiled code.
ReSym	1. Replication results match those presented in the paper. 2. ReSym's performance drops noticeably on our self-constructed dataset.
SymGen	1. Replication results are notably better than those reported in the paper. 2. SymGen's performance drops slightly on our self-constructed dataset.
GENNM	1. Replication results match those presented in the paper. 2. GENNM's performance drops slightly on our self-constructed dataset.
LLASM	We failed to run the artifact because the authors did not release the dataset or preprocessing procedure.
Duboue	We failed to run the artifact because the authors did not release the necessary resources, such as installation files, data, trained model, or scripts documentation.
DeMinify	We failed to run the artifact, as it only provides a project list and lacks data preparation instructions.
DeGuard	We failed to run the artifact, as the online tool consistently errors when processing APK files, including its own sample.
JSNice	1. Replication results match those presented in the paper. 2. JSNice supports only JavaScript ES5, but JavaScript ES6 is now more commonly used.
IDA Pro	1. IDA Pro executes efficiently and is easy to deploy. 2. IDA Pro's symbol name recovery performance is worse than dedicated approaches.
GPT	1. GPT-4o executes efficiently and is easy to deploy. 2. GPT-4o's symbol name recovery performance is worse than Claude Haiku and dedicated approaches.
Claude	1. Claude Haiku executes efficiently and is easy to deploy. 2. Claude Haiku achieves better name recovery performance than GPT-4o, but with much higher token cost.

5.1.1 Comparison Study.

For approaches/models that use native resources (i.e., CPU), we use time overhead as the metric of computational cost. Specifically, we measured the end-to-end time overhead under the same hardware, i.e., from the original binary to the final recovered names.

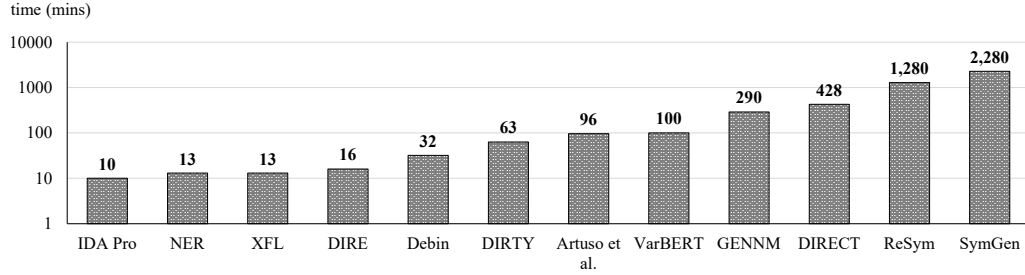


Fig. 1. Time for all evaluated artifacts

The time overhead for all evaluated artifacts (from shortest to longest) is shown in Figure 1. However, we argue that they are listed for reference purposes rather than comparative purposes because the time overhead is affected by many implementation-level factors, for example: (1) The artifacts differ in whether their preprocessing pipelines support multi-threading, which can accelerate preprocessing. (2) The artifacts differ in their inference hyperparameter settings, such as batch size, where larger batch sizes accelerate inference. (3) The artifacts differ in whether they support GPU acceleration, which can substantially accelerate inference.

References

- [1] Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. 2018. A general path-based representation for predicting program properties. In *Proceedings of the 2018 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI'18)*. 404–419.
- [2] Richard Baumann, Mykolai Protsenko, and Tilo Müller. 2017. Anti-proguard: Towards Automated Deobfuscation of Android Apps. In *Proceedings of the 2017 Workshop on Security in Highly Connected IT Systems (SHCIS '17)*. 7–12.
- [3] Rohan Bavishi, Michael Pradel, and Koushik Sen. 2018. Context2Name: A deep learning-based approach to infer natural variable names from usage contexts. *arXiv preprint arXiv:1809.05193* (2018).
- [4] Benjamin Bichsel, Veselin Raychev, Petar Tsankov, and Martin Vechev. 2016. Statistical deobfuscation of android applications. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security (CCS'16)*. 343–355.
- [5] Guillaume Breyton. 2023. *Neural inference of function names in stripped binaries*. Master's thesis. McGill University.
- [6] Guillaume Breyton, Mohd Saqib, Benjamin CM Fung, and Philippe Charland. 2024. BETAC: Bidirectional Encoder Transformer for Assembly Code Function Name Recovery. In *Proceedings of the 2024 International Conference on the Design of Reliable Communication Networks*. IEEE, 1–8.
- [7] Qibin Chen, Jeremy Lacomis, Edward J Schwartz, Claire Le Goues, Graham Neubig, and Bogdan Vasilescu. 2022. *Explore DirT*. Retrieved Dec. 17, 2025 from <https://dirtdirty.github.io/explorer.html>
- [8] Stelvio Cimato, Alfredo De Santis, and U Ferraro Petrillo. 2005. Overcoming the obfuscation of Java programs by identifier renaming. *Journal of systems and software (JSS)* 78, 1 (2005), 60–72.
- [9] The Standard Performance Evaluation Corporation. 2018. *SPEC CPU 2006 benchmark*. Retrieved Dec. 17, 2025 from <https://www.spec.org/cpu2006/>
- [10] Jingxuan He, Pesho Ivanov, Petar Tsankov, Veselin Raychev, and Martin Vechev. 2016. *DEBIN*. Retrieved Dec. 17, 2025 from <http://debin.ai>
- [11] Jingxuan He, Pesho Ivanov, Petar Tsankov, Veselin Raychev, and Martin Vechev. 2018. Debin: Predicting debug information in stripped binaries. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS'18)*. 1667–1680.
- [12] Yosuke Isobe and Haruaki Tamada. 2018. Are Identifier Renaming Methods Secure?. In *Proceedings of the 2018 IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD'18)*. 322–328.
- [13] Yosuke Isobe and Haruaki Tamada. 2018. Design and evaluation of the de-obfuscation method against the identifier renaming methods. *International Journal of Networked and Distributed Computing* 6, 4 (2018), 232–238.
- [14] Emily R Jacobson, Nathan Rosenblum, and Barton P Miller. 2011. Labeling library functions in stripped binaries. In *Proceedings of the 10th ACM SIGPLAN-SIGSOFT workshop on Program analysis for software tools (PASTE'11)*. 1–8.
- [15] Alan Jaffe. 2017. Suggesting meaningful variable names for decompiled code: a machine translation approach. In *Proceedings of the 2017 Joint Meeting on Foundations of Software Engineering (ESEC/FSE'17)*. 1050–1052.
- [16] Alan Jaffe, Jeremy Lacomis, Edward J Schwartz, Claire Le Goues, and Bogdan Vasilescu. 2018. Meaningful variable names for decompiled code: A machine translation approach. In *Proceedings of the 26th Conference on Program Comprehension (ICPC'18)*. 20–30.
- [17] Dongkwan Kim, Eunsoo Kim, Sang Kil Cha, Soeul Son, and Yongdae Kim. 2022. Revisiting binary code similarity analysis using interpretable feature engineering and lessons learned. *IEEE Transactions on Software Engineering (TSE)* 49, 4 (2022), 1661–1682.

- [18] Marie-Anne Lachaux, Baptiste Roziere, Marc Szafraniec, and Guillaume Lample. 2021. DOBF: A deobfuscation pre-training objective for programming languages. In *Proceedings of the 2021 International Conference on Neural Information Processing Systems (NeurIPS'21)*. 14967–14979.
- [19] Jeremy Lacomis, Alan Jaffe, Edward J Schwartz, Claire Le Goues, and Bogdan Vasilescu. 2018. Statistical machine translation is a natural fit for automatic identifier renaming in software source code. In *The Workshops at the 2019 AAAI Conference on Artificial Intelligence*. 771–774.
- [20] Ruigang Liang, Ying Cao, Peiwei Hu, Jinwen He, and Kai Chen. 2021. Semantics-recovering decompilation through neural machine translation. *arXiv preprint arXiv:2112.15491* (2021).
- [21] Nozima Murodova and Hyungjoon Koo. 2024. BinAdapter: Leveraging Continual Learning for Inferring Function Symbol Names in a Binary. In *Proceedings of the 2024 ACM Asia Conference on Computer and Communications Security (AsiaCCS'24)*. 1200–1213.
- [22] Naruaki Otsuki and Haruaki Tamada. 2022. Overcoming the obfuscation method of the dynamic name resolution. In *Proceedings of the 2022 International Conference on Software Engineering and Information Management (ICSIM'22)*. 118–124.
- [23] James Patrick-Evans, Lorenzo Cavallaro, and Johannes Kinder. 2020. Probabilistic naming of functions in stripped binaries. In *Proceedings of the 2020 Annual Computer Security Applications Conference (ACSAC'20)*. 373–385.
- [24] Steven Raemaekers, Arie Van Deursen, and Joost Visser. 2013. The maven repository dataset of metrics, changes, and dependencies. In *Proceedings of the 2013 Working Conference on Mining Software Repositories (MSR'13)*. 221–224.
- [25] Veselin Raychev, Martin Vechev, and Andreas Krause. 2015. Predicting Program Properties from 'Big Code'. In *Proceedings of the 2015 Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL'15)*.
- [26] Veselin Raychev, Martin Vechev, and Andreas Krause. 2019. Predicting program properties from 'big code'. *Communications of the ACM (CACM)* 62, 3 (2019), 99–107.
- [27] Zihan Sha, Hao Wang, Zeyu Gao, Hui Shu, Bolun Zhang, Ziqing Wang, and Chao Zhang. 2025. llasm: Naming functions in binaries by fusing encoder-only and decoder-only llms. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 34, 4 (2025), 1–22.
- [28] Alexandru-Gabriel SÎRBU. 2023. Deobfuscating JavaScript Code Using Character-Based Tokenization. *Studia Universitatis Babeş-Bolyai Informatica* (2023), 5–21.
- [29] Hieu Tran, Ngoc Tran, Son Nguyen, Hoan Nguyen, and Tien N Nguyen. 2019. Recovering variable names for minified code with usage contexts. In *Proceedings of the 2019 IEEE/ACM International Conference on Software Engineering (ICSE'19)*. 1165–1175.
- [30] Bogdan Vasilescu, Casey Casalnuovo, and Premkumar Devanbu. 2017. Recovering clear, natural identifiers from obfuscated JS names. In *Proceedings of the 2017 joint meeting on foundations of software engineering (ESEC/FSE'17)*. 683–693.
- [31] Hao Wang, Wenjie Qu, Gilad Katz, Wenyu Zhu, Zeyu Gao, Han Qiu, Jianwei Zhuge, and Chao Zhang. 2022. Jtrans: Jump-aware transformer for binary code similarity detection. In *Proceedings of the 2022 ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA'22)*.
- [32] Bing Xia, Jiabin Yin, Yunxiang Ge, and Ruinan Yang. 2023. A Binary Function Name Prediction Method Based on Variable Alignment and Translation Model. In *Proceedings of the 2023 International Conference on Machine Learning and Computer Application (ICMLCA'23)*. 757–761.
- [33] Xiangzhe Xu, Zhuo Zhang, Shiwei Feng, Yapeng Ye, Zian Su, Nan Jiang, Siyuan Cheng, Lin Tan, and Xiangyu Zhang. 2023. LmPa: Improving Decompilation by Synergy of Large Language Model and Program Analysis. *arXiv preprint arXiv:2306.02546* (2023).
- [34] Khaled Yakdan, Sergej Dechand, Elmar Gerhards-Padilla, and Matthew Smith. 2016. Helping johnny to analyze malware: A usability-optimized decompiler and malware analysis user study. In *Proceedings of the 2016 IEEE Symposium on Security and Privacy (SP'16)*. IEEE, 158–177.
- [35] Shih-Yuan Yu, Yonatan Gizachew Achamyelch, Chonghan Wang, Anton Kocheturov, Patrick Eisen, and Mohammad Abdullah Al Faruque. 2023. Cfg2vec: Hierarchical graph neural network for cross-architectural software reverse engineering. In *Proceedings of the 2023 IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP'23)*. 281–291.